Coursework: Designing Intelligent Agents Report

Module Code:	COMP4105
Module Title:	Designing Intelligent Agents
Student name:	Xin Zhang
Student ID number:	20343989
Submission date (Day Month Year):	10/05/2022

1. Background and problem statement

In this course, we will solve the problem of using different search algorithms to test agents' movement in a map environment. The basic project we used are The Pac-Man Projects from UC Berkeley and some resources from module COMP4105.

The main problem we aim to solve is planning Pac-Man's path so that he can eat as many dots on the map as possible in as little time as possible, and avoid ghosts. We use a score to measure the performance of the planning algorithm for Pac-Man.

$$\text{score} = n \cdot S_p - T - \text{isDead} \cdot \text{penalty}$$

Here, n is the number of dots eaten, S_p is the bonus score for eating a dot, T is the elapsed time, isDead is whether Pac-Man was eaten by a ghost or not, and the penalty is the score for being eaten by a ghost.

Our goal is to eat all dots while maximizing the score. The question is to compare the performance of A* algorithm, genetic algorithm and simulated annealing algorithm in task performance.

2. Algorithm

In this project, map data is quantized into a matrix of 2 dimension. For each position, Pac-Man can choose to move in one of four directions: East, South, west, or north. However, the map is bounded and there are wall markers within the map, which means Pac-Man cannot move in the direction of the wall boundary.

2.1 A* algorithm

The A* algorithm is the first path planning algorithm we used on the Pac-Man project. The core idea of A* algorithm is to calculate the priority of the four directions of the current seat by heuristic function and move to the direction with the lowest priority. The priority function is shown below:

$$f(n) = g(n) + h(n) + \sum ghosts(n)$$

Among them:

1. $f(n)$ is the comprehensive priority of node n. When we choose the next node to traverse, we always select the node with the highest overall priority (lowest value).
2. $g(n)$ is the cost of the distance between node n and the starting node.
3. $h(n)$ is the estimated cost of node n from the ending node, H (n) is the estimated cost of node n to the ending node and the heuristic function of A* algorithm also. Heuristic functions are discussed in detail below.
4. $\sum ghosts(n)$ represents the weight of the ghost, which is high when the ghost is around Pac-Man and low when the ghost is around Pac-Man (if the ghost i is eatable, the $ghost(i)$ will be negative value). Since there's more than one ghost, it's a sum.

A* algorithm selects the node with the lowest $f(n)$ value (highest priority) from the priority queue as the next node to be traversed.

As mentioned above, heuristics affect the behavior of the A* algorithm.

In extreme cases, when the heuristic function $h(n)$ is always 0, $g(n)$ and $\sum ghosts(n)$ determine the priority of nodes, and then the algorithm degenerates into Dijkstra algorithm.

If $h(n)$ is always less than or equal to the cost of node n to the destination, the shortest path can be found by A* algorithm. However, the smaller the value of h(n) is, the more nodes will be traversed by the algorithm and the slower the algorithm will be.

If $h(n)$ is exactly equal to the cost of node n to the end node, then A* algorithm will find the best path between two points quickly. However, this can't be done in all scenarios. Because it's hard to figure out exactly how far we are until we reach the finish line.

If the value of $h(n)$ is greater than the cost of node n to the destination, A* algorithm cannot guarantee to find the shortest path, but it is faster.

On the other hand, if $h(n)$ is much greater than the value of $g(n)$, then it will be only $h(n)$ working, and this becomes a best-first search.

From the above information, it is known that the speed and accuracy of the algorithm can be controlled by adjusting the heuristic function. In some cases, we may not necessarily need a shortest path, but rather want to find one as quickly as possible. Therefore, A* algorithm is A flexible method.

2.2 Genetic algorithm (GA)

The second algorithm we used is the genetic algorithm. Genetic algorithm is a kind of hard coding.

Genetic algorithm is to simulate the evolution process under the principle of "survival of the fittest" in the magical nature. Good genes have more chances to reproduce. In this way, as the reproduction progresses, the biological population will converge towards a trend. Genetic hybridization and variation in the process of biological reproduction will provide a better genetic sequence for the population, so that the population's reproductive tendency will be each generation better than the previous generation, rather than being limited to the best genes of the ancestors. A program can simulate this process to get an optimal solution to a problem (but not always). To use this process to solve the problem, the limitation is to construct an initial genome, initialize each gene with a fitness score (a measure of how good or bad the gene is), and then select two fathers from the original genome (selected using a roulette algorithm based on the fitness score) to reproduce. Based on a certain rate of hybridization (the probability that the father will cross) and mutation (the probability that the child will change), the two fathers will make two child genes, and then these two genes will be put into the population, where they will reproduce, and the process of reproduction will be repeated until the population converges or the fitness score is

maximized.

In our pathfinding algorithm, because it's hard-coded, the gene pool is the direction Pac-Man goes at each location.

The main process of the GA is below:

1. Examine each gene's problem-solving ability and quantify its value
2. Select a gene from the current memory bank as the parent. The principle of selection is: the stronger the solution ability, the greater the probability of selection.
3. The selected two are crossbred according to the hybrid rate to produce progeny.
4. The offspring are mutated according to the rate of mutation
5. Repeat step 2, 3, 4 until a new generation is created

First, we give all the positions on the map a random and valid direction, and when Pac-Man is in that position, he will move in the pre-coded direction at the next time.

Next, we will go into detail on how hybridization and mutation are implemented in our hard-coded problem.

In the process of hybridization, we first experimented with the hard coding of the population of this generation. The parents with excellent performance (high score) were randomly combined with their horizontal and vertical coding according to a certain component to obtain the offspring.

In the mutation process, we randomly and slightly mutate the code of the progeny obtained in the previous step, and repeat the process until we get the next generation population.

2.3 Simulated annealing algorithm

The third heuristic algorithm we used is the simulated annealing. The inspiration of simulated annealing algorithm comes from the solid annealing principle in physics:

when the solid is heated, the particles inside the solid become disordered with the temperature going up, and the internal energy accumulate continuously; When cooling slowly, the internal particles are gradually ordered, reaching equilibrium state at various temperatures, and reaching ground state at common temperature, where the internal energy is decreased to a minimum value. In computer language, it is described as follows: in the process of continuous iteration of the function, a new solution worse than the current one has a reasonable chance of being accepted. Therefore, it is possible to get out of the local optimal solution and achieve the global optimal solution.

The operation of this algorithm for map coding is similar to that of genetic algorithm, which will not be explained in detail here.

We first define a temperature and decrease the temperature in each iteration. We stop the iteration when the temperature is less than a certain value.

Next, as we iterate, we pick the codes with the best scores in each iteration and randomly change some of them. In this process, if the experimental result of the new code is better than the last one, we accept it, otherwise we will accept the new bad code with a certain probability.

3. Experiments

In this project, we test the performance of our algorithm in *runGames.py*. The map layout we used is "mediumClassic". A* algorithm is implemented in *newAgents. py* and we can test it in *runGames.py.* In each step of our A* algorithm, we calculate the priority of all directions based on all the dots, and move to the direction with the highest priority, the lower the penalty score. Unlike traditional A*, we do not record the path, which means that our algorithm is no longer suitable for maze-finding. This is because Pac-Man may have to backtrack after eating a dot, or he may have to backtrack after encountering a ghost while eating a dot.

We test this algorithm in two ghosts walking at random state, and we simulate it 10

times. The results are as follows:

Pac-man status	Score
Pacman emerges victorious!	2078
Pacman emerges victorious!	1914
Pacman emerges victorious!	1649
Pacman emerges victorious!	1729
Pacman emerges victorious!	1720
Pacman died!	466
Pacman emerges victorious!	1660
Pacman died!	-50
Pacman emerges victorious!	1326
Pacman emerges victorious!	1685
Pacman emerges victorious!	1715

The average score is: 1423.2. (Due to the randomness of the ghost, the score is only referential)

As you can see from the results above, Pac-Man's victory score fluctuates around 1600, but there are rare cases where Pac-Man is killed by a ghost. This is because there are so many ghosts that they can strike from both sides, so Pac-Man gets eaten. But overall, the algorithm performs well.

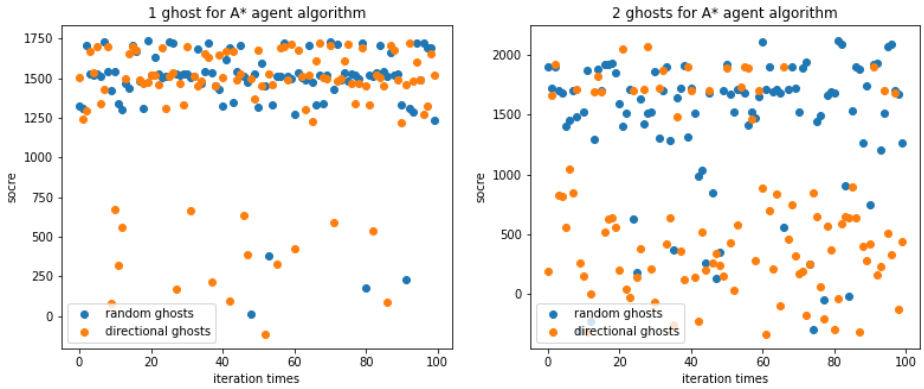

Figure 1. The score distribution of A* algorithm in the case of two different types of ghosts (directional ghosts and random walk ghosts) and different numbers of ghosts.

The figure above shows the score distribution of A* algorithm for different types of ghosts (directional ghosts, random walk ghosts) when the number of ghosts is equal to 1 or 2 respectively. It can be seen that when the number of ghosts is equal to 1, the ghost type has little influence on the performance of A* algorithm, but when the number of ghosts is equal to 2, the death probability of Pac-Man greatly increases. This is due to the fact that the two types of ghosts can "work together" to surround Pac-Man when there are more of them, making Pac-Man unable to escape the surrounding net and resulting in a certain death situation, while the directional ghosts are more cooperative and Pac-Man dies more often.

Considering the performance of genetic algorithm and simulated annealing algorithm, the learning ability of genetic hard-coding algorithm is extremely limited due to the randomness of ghost. In the medium map, Pac-Man can eat dots on the map to a certain extent, but the probability of death is very high and often get stuck on the movement, resulting in low scores. Here we will simulate many times, and the simulation results are shown in the figure below.

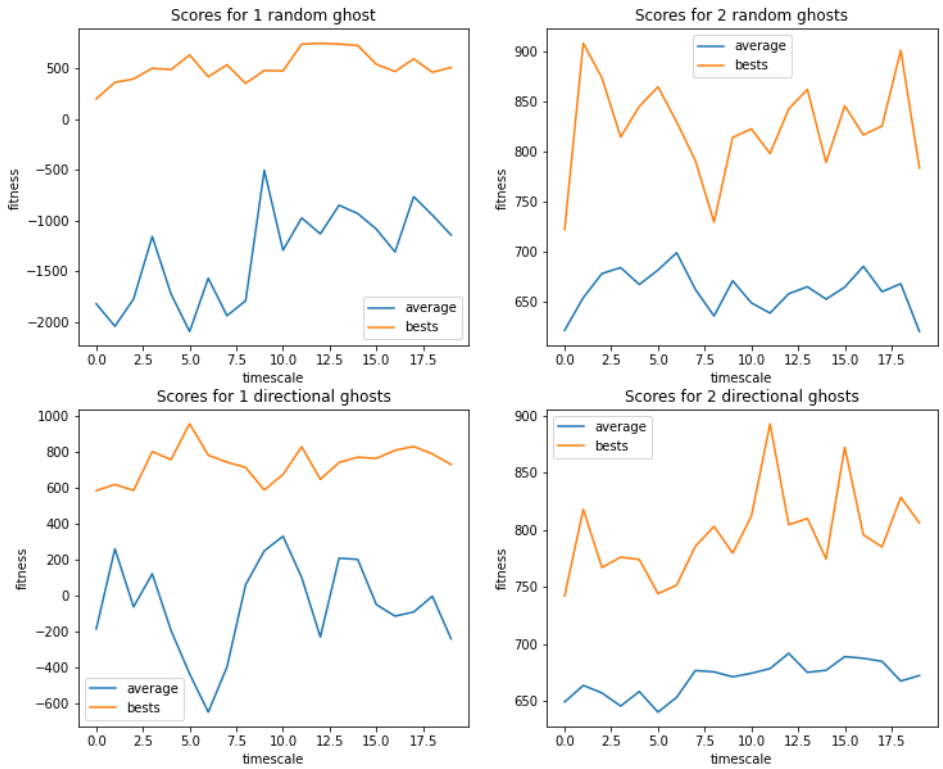

Figure 2. Simulation results of genetic algorithm. The four graphs show the best fitness (orange lines) and average fitness (blue lines) for genetic algorithms in different situations.

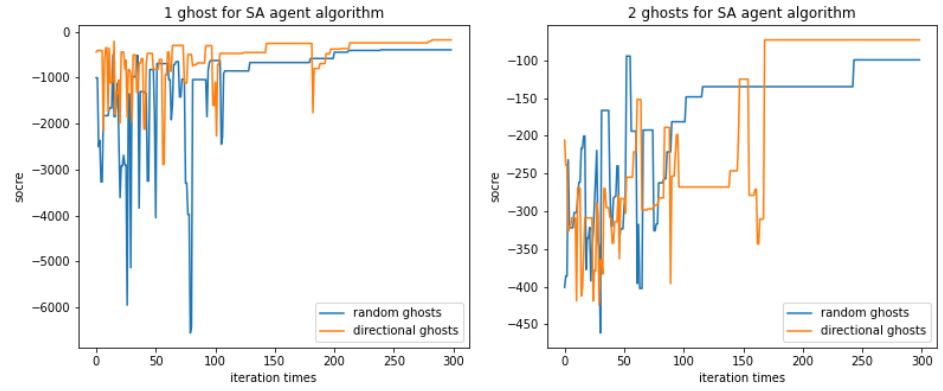

Figure 3. Experimental results of simulated annealing. The graph shows the relationship between training times and scores. The ordinate score of the graph is the score of the original running result, without adding 1000 score.

From the above results, although these two algorithms can deal with and learn the agent path finding problem to a certain extent. Genetic algorithm performs better with a single directed ghost than with a random ghost, it also can be seen that a slight increase in score as an increase in ghost numbers. The simulated annealing algorithm scored better with two ghosts, and it performs better against a directed ghost in the case of one ghost.

However, due to the randomness of ghost movement, the learning ability of these two methods is very limited. The problem can be considered a real-time game, and any pre-determined method is difficult to deal with such an online problem.

In the case of simple map, the above two algorithms can also have a good performance. But it is obvious that A* algorithm is better than genetic algorithm and simulated annealing algorithm in this environment.

4. Conclusion

This work uses A custom A* algorithm, genetic hard-coding algorithm and simulated annealing hard-coding algorithm to solve the agent path finding problem: Pac-Man gets more points by eating more beans without being eaten by ghosts. This algorithm is a real-time strategy, which needs to be solved by online algorithm. From the above results, A* has the best effect, basically eating all the beans in the shortest time, and the probability of death is very low. This improved version of A* cannot be used for maze finding because it does not record historical paths. In our algorithm, we use predictive next steps to prevent the agent from swinging back and forth between the two positions and causing the strategy to get bogged down. Overall, A* performs well. For genetic algorithm and simulated annealing algorithm, due to the randomness of the ghost and the hard-coding characteristics of the algorithm, we cannot obtain the state of the ghost immediately and use this factor to change the algorithm. We can only judge whether the coding method is good by the score index of each running result, which has great limitations.

In the future work, we will explore more online algorithms and try to apply artificial intelligence algorithms in this real-time strategy, possibly by means of back propagation of learning weights to enable the agent to deal with more situations and complete more tasks.